



Welcome to the Python Course!

This document compiles the foundational topics learned during the first week. It serves as a comprehensive study guide, detailing key concepts, syntax, and basic practical examples. All explanations and code blocks are derived directly from the weekly log repository.

Table of Contents / Topics Covered:

- 1. Variables
- 2. Datatypes
- 3. Operators
- 4. If-Else Conditions
- 5. Looping Statements
- 6. Lists
- 7. Dictionary
- 8. Practice Questions

1. Variables

In Python, variables are used to store data, and they do not require explicit declaration. Python automatically determines the data type based on the value assigned.

```
# Assigning values to variables
x = 5
y = "Hello"
print(x)
print(y)
```

2. Datatypes

Python has several standard data types to store different types of data. Here are the core data types covered in Week One:

Integer (int)

Numbers without decimal points (positive, negative, or zero).

```
a = 10
b = -20
```

Float (float)

Numbers with decimal points.

```
f1 = 1.35
pi = 3.14
```

String (str)

A collection of characters enclosed within a pair of single (') or double (") quotes.

```
word = "Hello"
sentence = 'Hello World'
```

Boolean (bool)

A logical value representing either True or False.

```
adult = True
male = False
```

3. Operators

Operators are special symbols used to perform operations on variables and values.

Arithmetic Operators

Used to perform mathematical operations:

- **+** : Addition (adds two values)
- **-** : Subtraction (subtracts second value from first)
- ***** : Multiplication (multiplies two values)
- **/** : Division (returns float value of division)
- **//** : Floor Division (truncates decimal part of division)
- **%** : Modulus (returns the remainder of division)
- ****** : Exponentiation (raises first value to power of second)

```
a = 20
b = 10

print(a + b) # Output: 30
print(a - b) # Output: 10
print(a * b) # Output: 200
print(a / b) # Output: 2.0
print(a // b) # Output: 2
print(a % b) # Output: 0
print(a ** b) # Output: 1024000000000000000000
```

Relational (Comparison) Operators

Used to compare two values. Returns either True or False.

- **>** : Greater than
- **<** : Less than
- **==** : Equal to
- **>=** : Greater than or equal to
- **<=** : Less than or equal to
- **!=** : Not equal to

```
a = 0
b = 1

print(a > b) # Output: False
print(a < b) # Output: True
print(a == b) # Output: False
print(a >= b) # Output: False
print(a <= b) # Output: True
print(a != b) # Output: True
```

Logical Operators

Used to combine conditional statements.

- **and** : Returns True if both statements are true
- **or** : Returns True if one of the statements is true
- **not** : Reverses the result, returns False if the result is true

```
a = True
b = False

print(not a) # Output: False
print(a and b) # Output: False
print(a or b) # Output: True
```

Membership Operators

Used to test if a sequence (like a string or list) is present in an object.

- **in** : Returns True if value is present in the sequence
- **not in** : Returns True if value is not present in the sequence

```
word = "Hello"

print("e" in word) # Output: True
print("e" not in word) # Output: False
```

Assignment Operators

Used to assign values to variables.

- **=** : Simple assignment
- **+=** : Add and assign (a += b is equivalent to a = a + b)
- **-=** : Subtract and assign
- ***=** : Multiply and assign
- **/=** : Divide and assign

```
a = 10
a = a + 15 # a becomes 25
print(a)

a = 10
a *= 5 # a becomes 50
print(a)
```

4. If-Else Conditions

Conditional statements allow decision-making in code based on specified boolean conditions.

```
if condition1:
    # block executed if condition1 is True
elif condition2:
    # block executed if condition2 is True
else:
    # block executed if all conditions above are False
```

Practical If-Else Examples

Example 1: Check Adult Status

```
age = int(input("Please enter your age: "))
if age >= 18:
    print("Adult")
else:
    print("Not an Adult")
```

Example 2: Check Even or Odd Number

```
num = int(input("Enter a number: "))
if num % 2 == 0:
    print("Even")
else:
    print("Odd")
```

Example 3: Check Exam Result (Pass / Fail)

```
passing_marks = 35
marks_obtained = int(input("Enter your marks: "))

if marks_obtained <= passing_marks:
    print("Sorry. You have Failed!")
else:
    print("Yayy! You have passed!")
```

5. Looping Statements

Loops allow us to execute a block of code repeatedly without writing repetitive lines of code.

Types of Loops

For Loop: A for loop is used to iterate over a sequence (such as a range() object, string, or list).

```
# Iterating over numbers from 0 to 5
for num in range(0, 6):
    print(num, end=" ", " ")
```

While Loop: A while loop executes a block of statements repeatedly as long as the specified condition remains True.

```
# Printing numbers from 0 to 5 using a while loop
num = 0
while num <= 5:
    print(num)
    num += 1
```


Loop Control Statements

break Statement: The break statement terminates the loop execution immediately when encountered.

```
# Terminate loop as soon as the target letter is found
word = "apple"
letter = "p"
found = False

for i in range(0, len(word)):
    print(f"Checking! {word[i]}")
    if word[i] == letter:
        found = True
        print(f"Found at index: {i}")
        break # Exit loop immediately
```

continue Statement: The continue statement skips the rest of the current loop iteration and moves directly to the next iteration.

```
# Skip even numbers and print only odd numbers
end = 10
for i in range(0, end + 1):
    if i % 2 == 0:
        continue # Skip even numbers
    print(i)
```

6. Lists

A list is a built-in, ordered, and mutable sequence in Python. It can store elements of various data types, including other lists.

List Definition & Creation

```
# Creating an empty list using list() constructor
nums = list()
print(nums) # Output: []

# Creating a list using square brackets
nums = [1, 2, 3]
print(nums) # Output: [1, 2, 3]
```

Key List Operations & Methods

Appending Elements: Adds an element to the end of the list using `append()`.

```
nums = [1, 2, 3]
nums.append(4)
print(nums) # Output: [1, 2, 3, 4]

# Appending a list inside a list (nested list)
nums.append([5, 6])
print(nums) # Output: [1, 2, 3, 4, [5, 6]]
```

Deleting Elements: Using `pop()` (index-based) or `remove()` (value-based).

```
# Using pop() to remove the last element
nums = [1, 2, 3, [4, 5, 6]]
nums.pop()
print(nums) # Output: [1, 2, 3]

# Using remove() to remove a specific item
nums = [1, 2, 3, [4, 5, 6]]
nums.remove([4, 5, 6])
print(nums) # Output: [1, 2, 3]
```

Inserting Elements: Inserts an element at a specified index using `insert()`.

```
nums = [1, 2, 3]
# Insert value 4 at index 1
nums.insert(1, 4)
print(nums) # Output: [1, 4, 2, 3]
```

Sorting & Reversing: Using `sort()` and slicing `[::-1]`.

```
nums = [1, 6, 2, 5, 3, 4]

# Ascending order
nums.sort()
print(nums) # Output: [1, 2, 3, 4, 5, 6]

# Descending order
nums.sort(reverse=True)
print(nums) # Output: [6, 5, 4, 3, 2, 1]

# Reversing a list using slicing
nums = [1, 5, 2, 4, 3]
reversed_nums = nums[::-1]
print(reversed_nums) # Output: [3, 4, 2, 5, 1]
```

7. Dictionary

A dictionary is an ordered (as of Python 3.7) and mutable collection of key-value pairs. Keys must be unique and immutable (such as strings, numbers, or tuples), while values can be of any data type.

Creating a Dictionary

```
student = {  
    "s_id": 100,  
    "name": "Ajay",  
    "marks": [85, 90, 92],  
    "contact": {"dad": 1234567890, "mom": 9876543210}  
}
```

Key Dictionary Operations

Insertion & Updation: New keys are inserted or existing keys updated using square bracket notation.

```
d = {}  
  
# Inserting new keys  
d['name'] = "Ajay"  
d['age'] = 20  
  
# Updating an existing key  
d['age'] = 21  
print(d) # Output: {'name': 'Ajay', 'age': 21}
```

Deletion: Removing key-value pairs using del or .pop().

```
d = {'name': 'Ajay', 'age': 21, 'city': 'Mumbai'}  
  
# Using del keyword  
del d['city']  
  
# Using pop() method  
age = d.pop('age')  
print(age) # Output: 21  
print(d) # Output: {'name': 'Ajay'}
```

Dictionary Methods & Iteration

Keys & Values View: Retrieve views of keys and values using `.keys()` and `.values()`.

```
d = {'name': 'Ajay', 'age': 21, 'role': 'Student'}

# Get all keys
print(d.keys()) # Output: dict_keys(['name', 'age', 'role'])

# Get all values
print(d.values()) # Output: dict_values(['Ajay', 21, 'Student'])
```

Practical Example: Anagram Problem

We can check if two words are anagrams by building and comparing character frequency dictionaries.

```
w1 = input("Enter word 1: ")
w2 = input("Enter word 2: ")

d1 = {}
d2 = {}

# Populate frequency dictionary for word 1
for ch in w1:
    d1[ch] = d1.get(ch, 0) + 1

# Populate frequency dictionary for word 2
for ch in w2:
    d2[ch] = d2.get(ch, 0) + 1

# Compare the two dictionaries
if d1 == d2:
    print("Anagram")
else:
    print("Not Anagram")
```

Practice Questions for Week -1

1. Count Positive, Negative, and Zero

Problem Statement:

Write a Python program that counts how many numbers in a list are positive, negative, and zero.

Example:

Input:

```
numbers = [4, -2, 0, 7, -5, 0, 9, -1]
```

Expected Output:

```
Positive: 3  
Negative: 3  
Zero: 2
```

2. Word Frequency Counter

Problem Statement:

Write a Python program that counts how many times each word appears in a list. Store the result in a dictionary where the key is the word and the value is its frequency.

Example:

Input:

```
words = ["apple", "banana", "apple", "orange", "banana", "apple"]
```

Expected Output:

```
{  
    "apple": 3,  
    "banana": 2,  
    "orange": 1  
}
```

3. Student Marks

Problem Statement:

You are given a dictionary containing student names as keys and their marks as values. Write a Python program to determine whether each student has Passed or Failed. A student passes if their marks are 50 or above; otherwise, they fail.

Example:**Input:**

```
marks = {  
    "Alice": 90,  
    "Bob": 65,  
    "Charlie": 82,  
    "David": 45  
}
```

Expected Output:

```
Alice : Passed  
Bob : Passed  
Charlie : Passed  
David : Failed
```

4. Shopping Cart Total

Problem Statement:

A shopping cart is represented as a dictionary where the key is the item name and the value is its price. Write a Python program to: Calculate the total bill. If the total bill is greater than 150, print "Discount Applied". Otherwise, print "No Discount".

Example:**Input:**

```
cart = {  
    "Milk": 40,  
    "Bread": 30,  
    "Eggs": 60,  
    "Butter": 80  
}
```

Expected Output:

```
Total Bill: 210  
Discount Applied
```